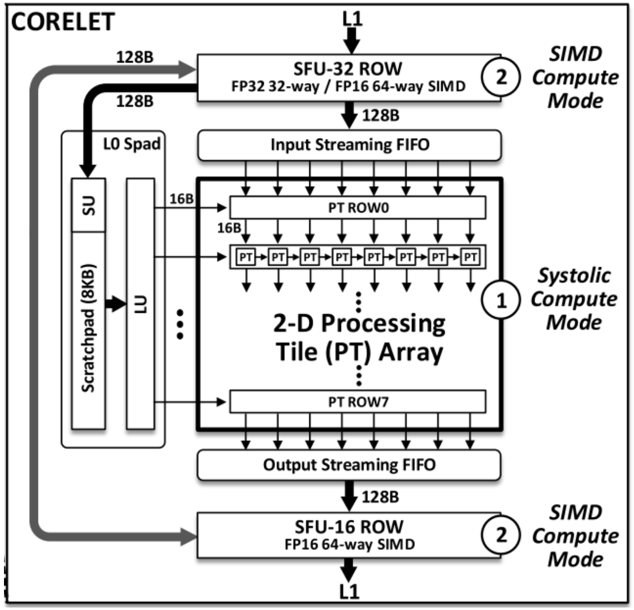
ECE260B Winter

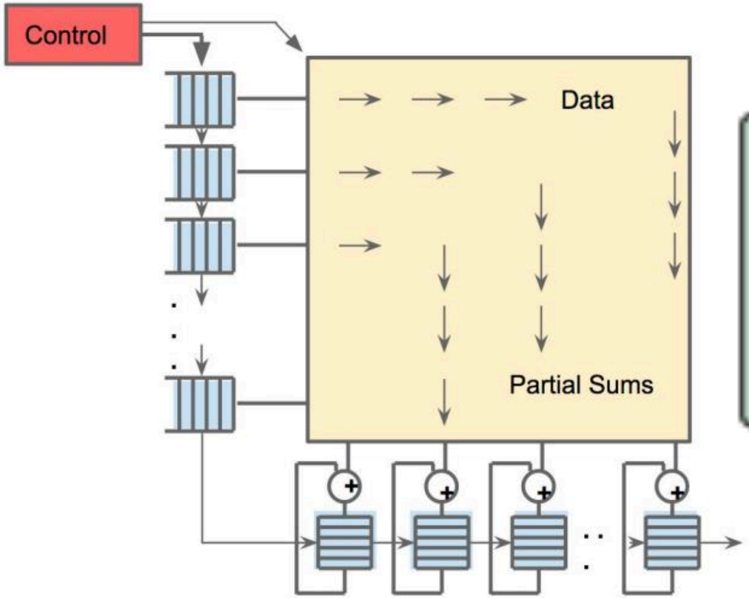
Project Description

Prof. Mingu Kang

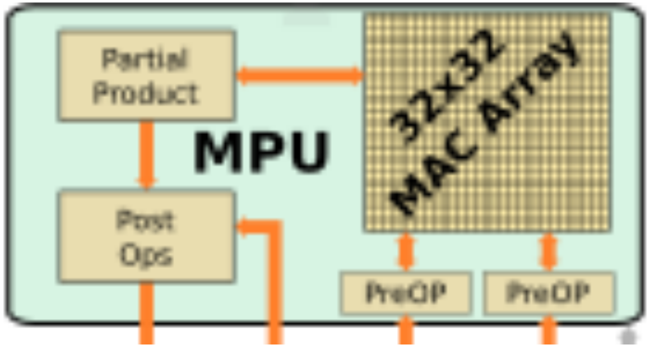
2D Systolic-based Architecture



IBM Rapid



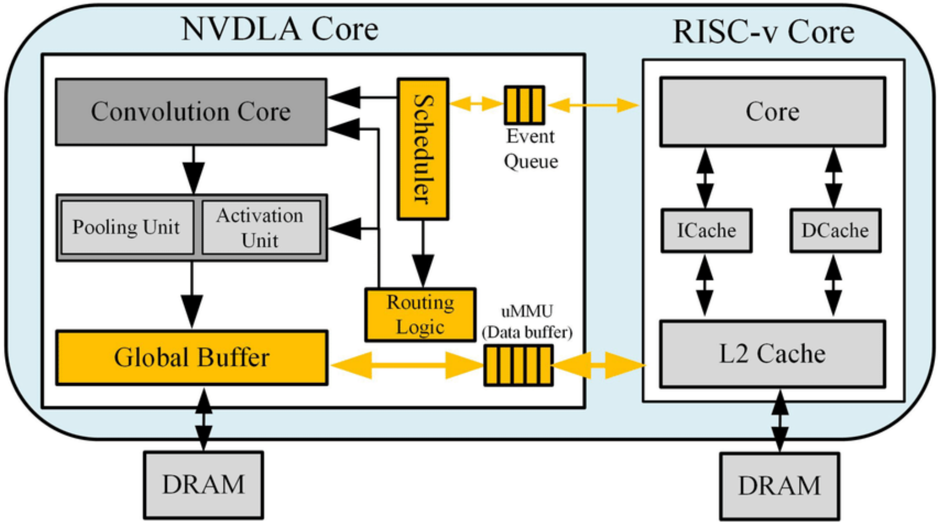
Google TPU



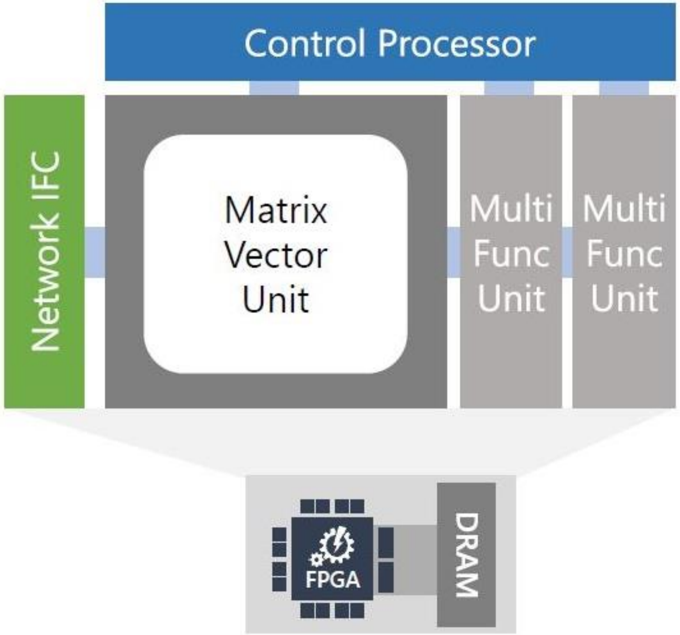
Intel Nervana

J.Oh, “A 3.0 TFLOPS 0.62V Scalable Processor Core for High Compute Utilization AI Training and Inference“
https://arxiv.org/pdf/1704.04760.pdf
https://fuse.wikichip.org/news/3270/intel-axes-nervana-just-two-months-after-launch/

1-D Vector Processor based Architecture



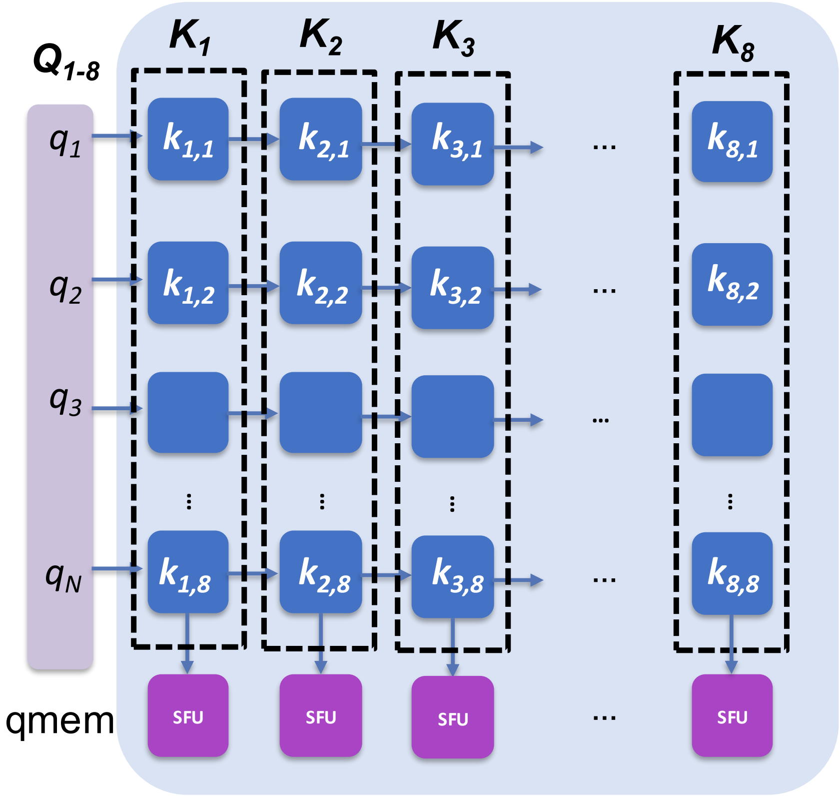
NVIDA NVDLA



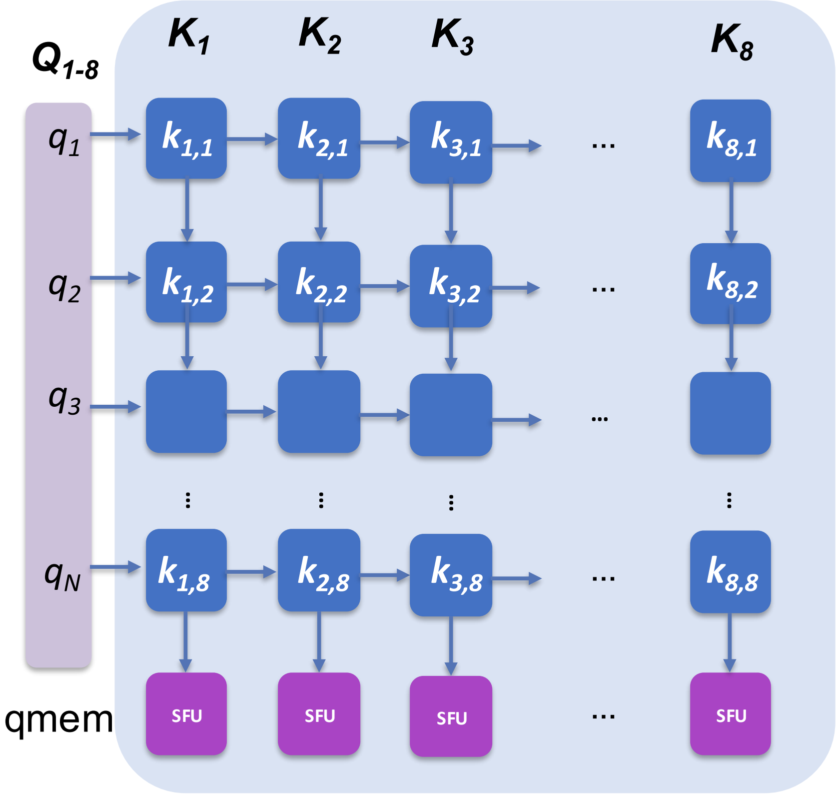
Microsoft Brainwave

https://www.researchgate.net/figure/Elastic-NVDLA-and-RISC-V-implement-DNNGuard-architecture_fig4_339920895
https://www.servethehome.com/microsoft-shows-off-project-brainwave-persistent-inferencing-from-fpga-cache/microsoft-brainwave-soft-dpu-architecture/

Architecture



1-D vector processor architecture



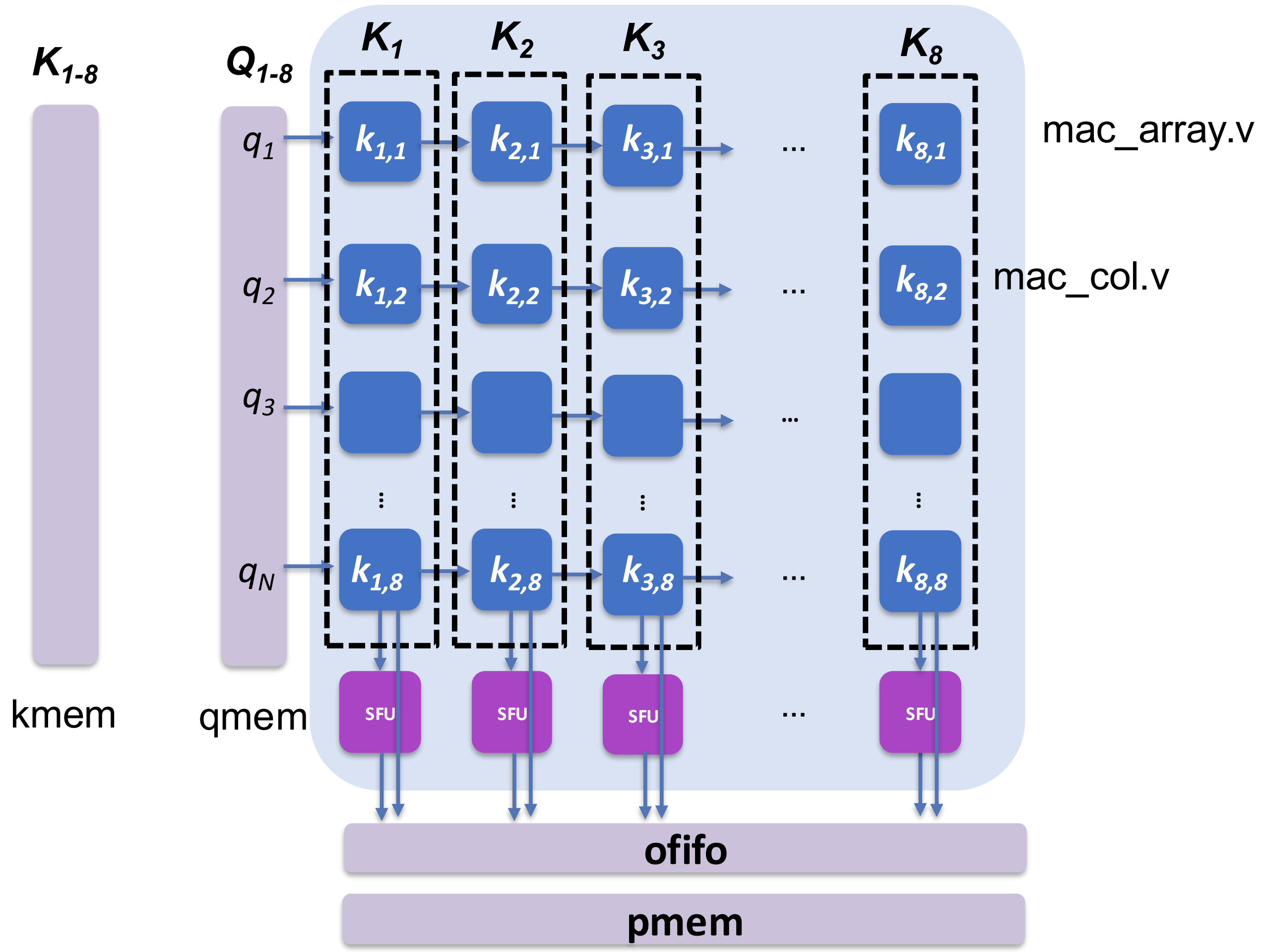
2D-systolic array architecture

Attention Mechanism in Transformer Models

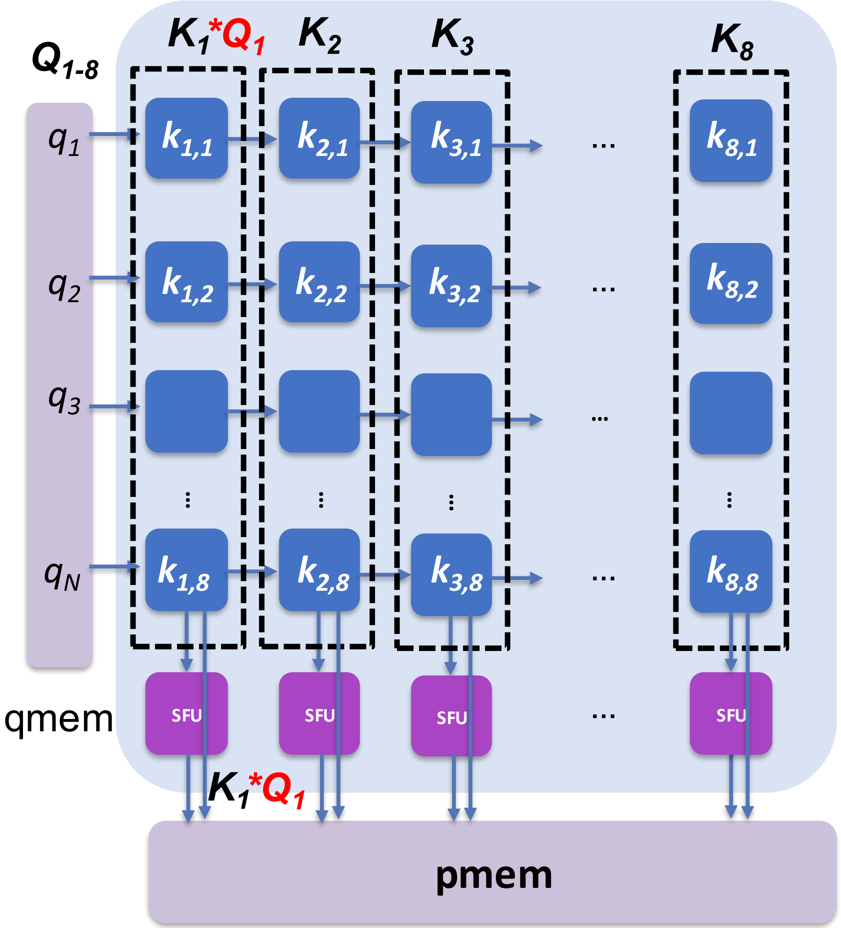
	Morril's	wife	, Ellie,	hugged	her	Sister.
Query (Q)	Q_1	Q_2	Q_3	Q_4	Q_5	Q_6
Key (K)	K_1	K_2	K_3	K_4	K_5	K_6
Value (V)	V_1	V_2	V_3	V_4	V_5	V_6
Score (= $Q \times K$)	0.1	0.3	0.35	0.2	0.15	0.1
Prob (= Softmax[$QK/\sqrt{d}$])	0.01	0.41	0.45	0.02	0.1	0.01
Prob * Value (V)	$0.01V_1$	$0.41V_2$	$0.45V_3$	$0.02V_4$	$0.1V_5$	$0.01V_6$
Sum			Z_3			

- Q, K, V are vectors (not scalar)
- Note softmax results are unsigned
- For simplicity, we use normalization instead of softmax
- Generally Q×K requires lower bit precision while Prob×V requires higher bit precision

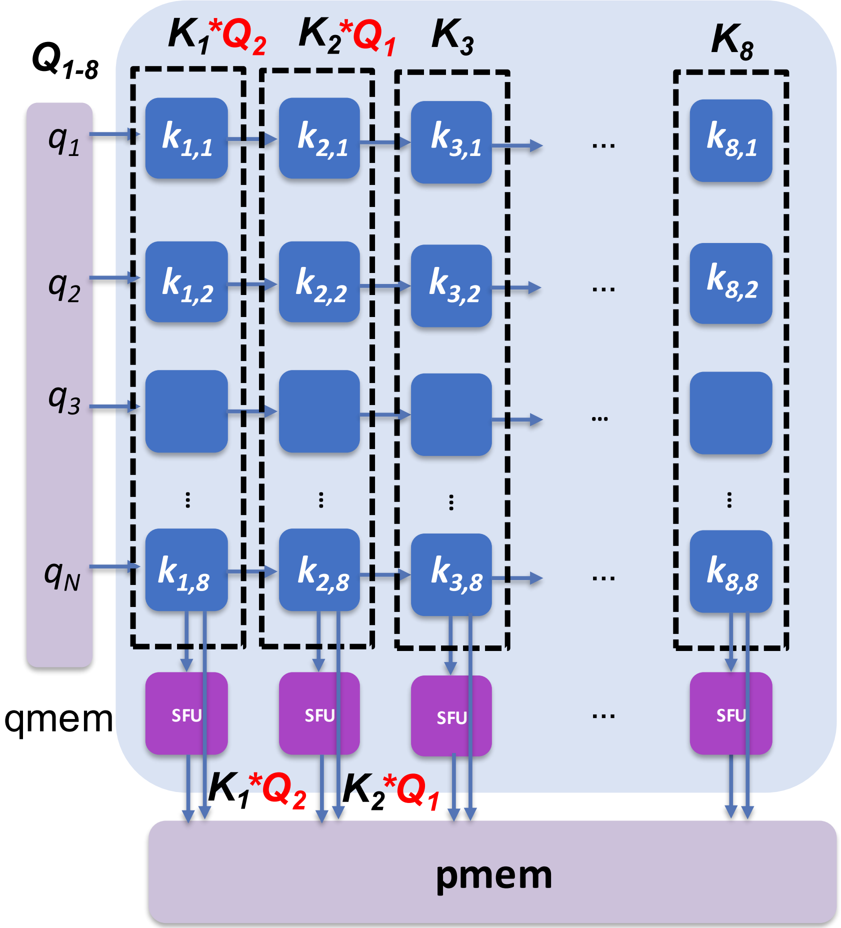
Architecture



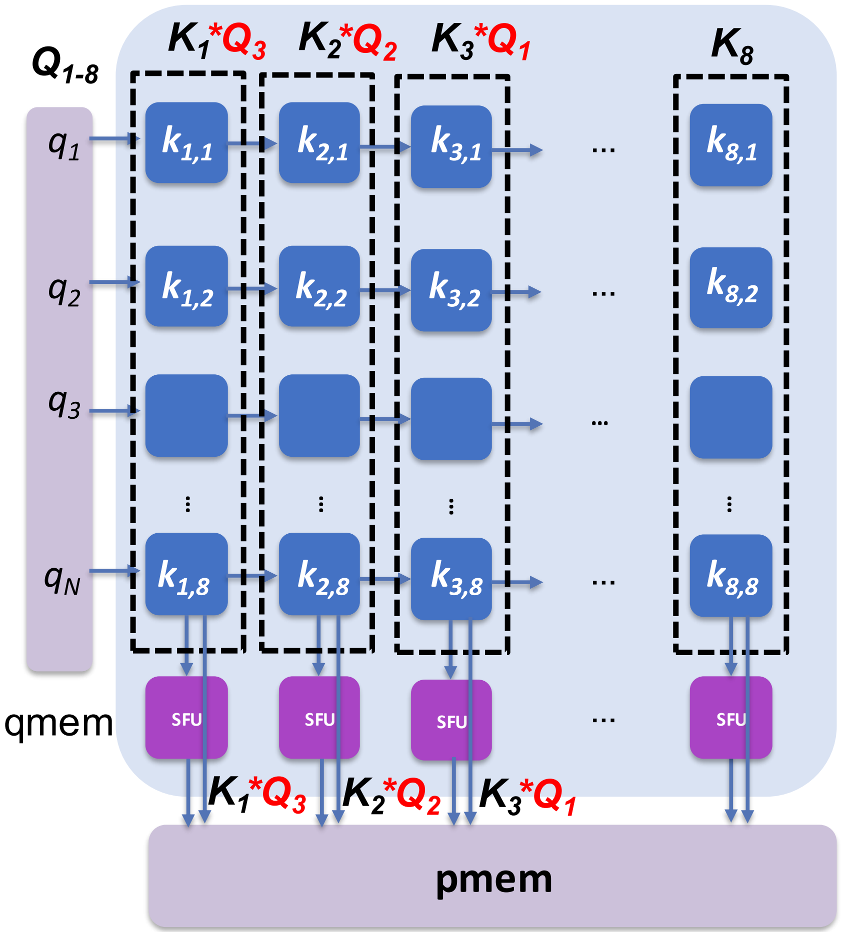
Computation



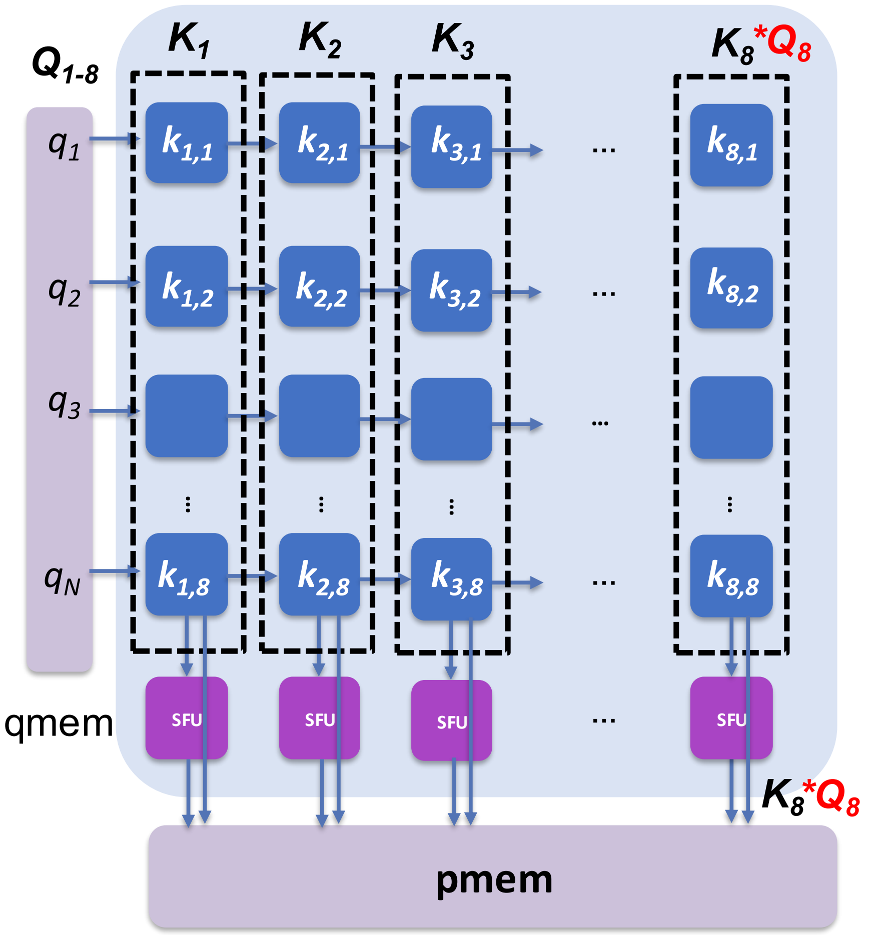
Computation



Computation



Computation



Processing Stage

- Stage1: memory write stage

  - qmem & kmem: fill both memory with “qdata.txt” and “kdata.txt”

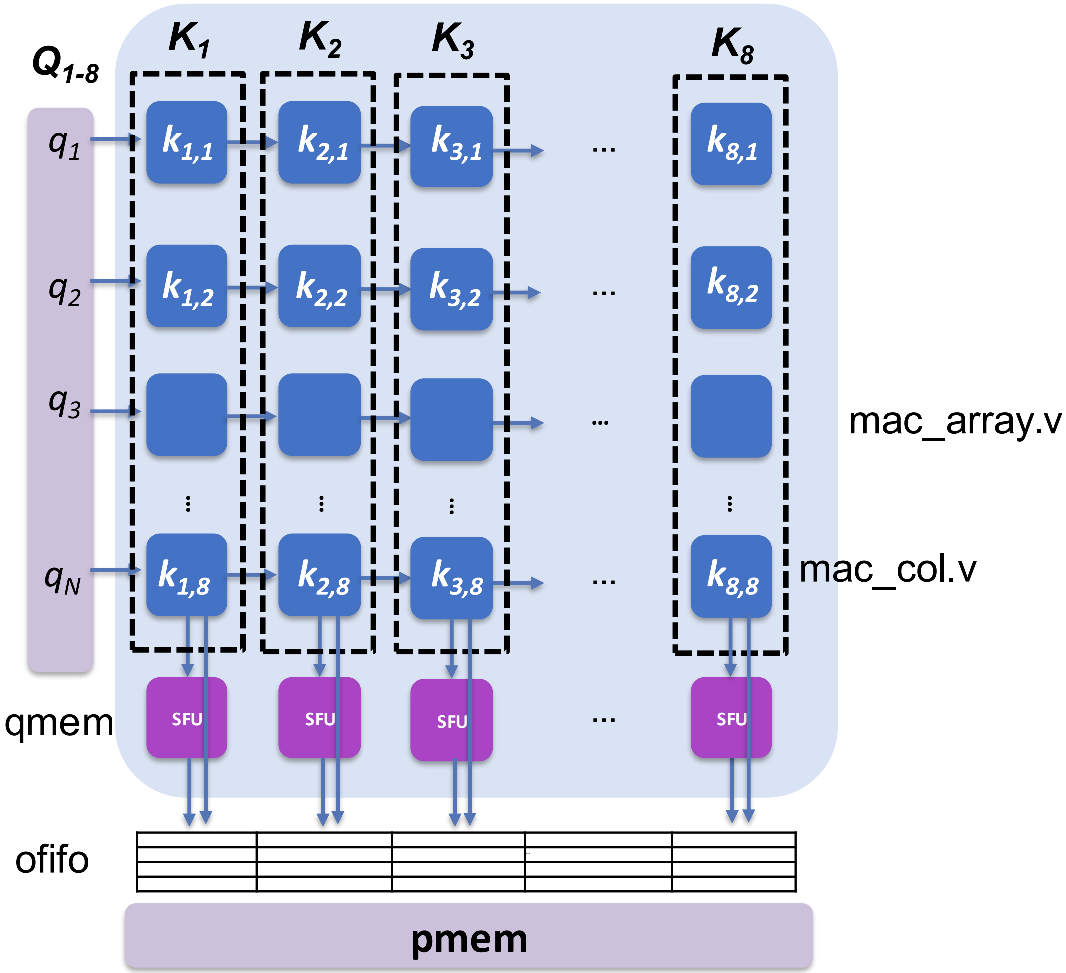
- Stage2: k-data loading stage

  - move the data from kmem to process’s local register

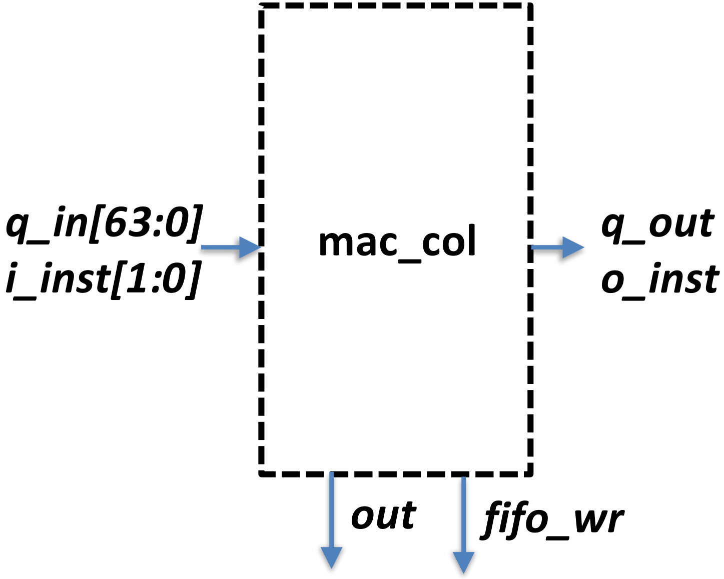
- Stage3: execute

  - move the data from qmem to input of the processor

More Detail

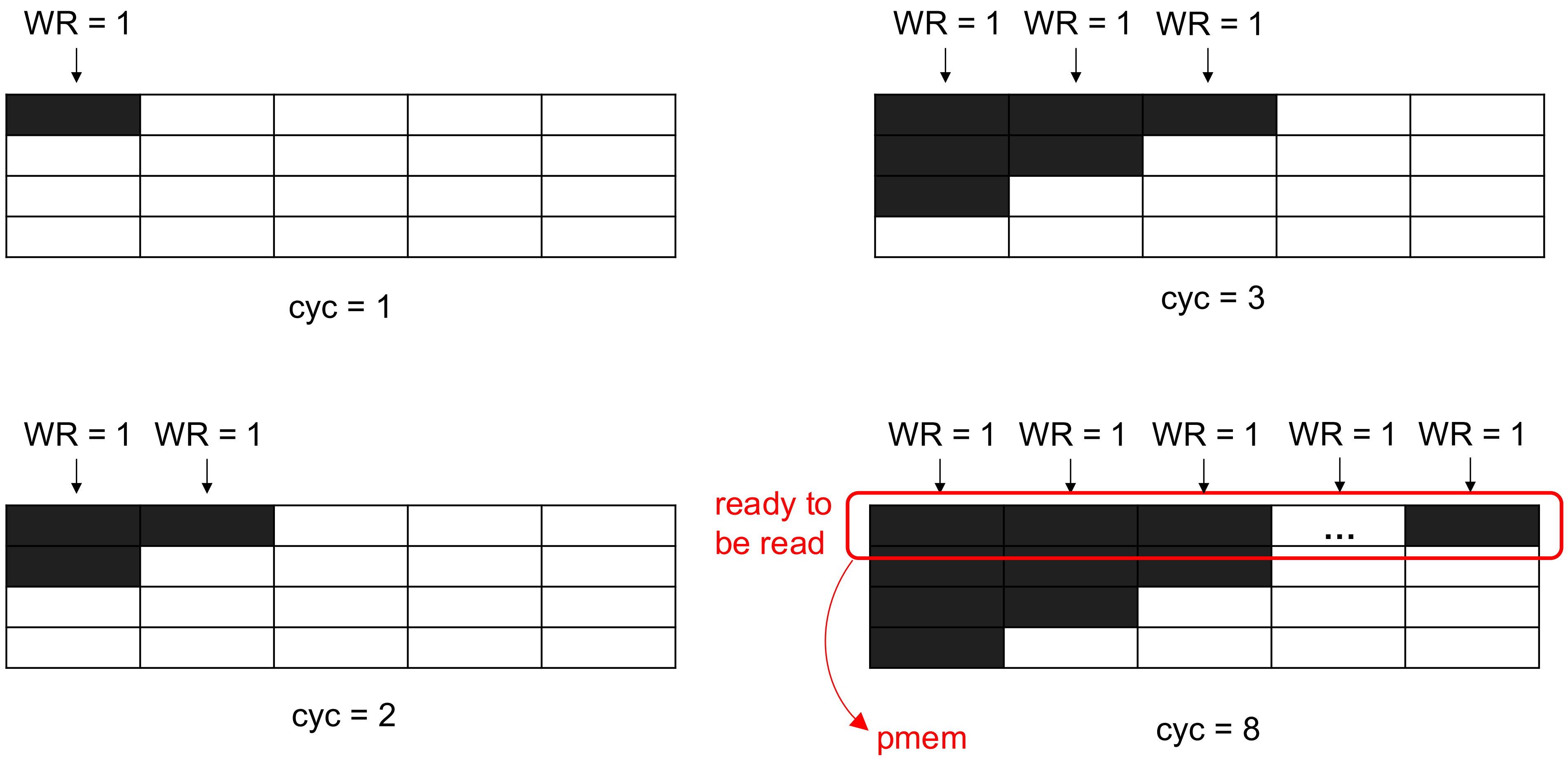


Q_{1-8}
K_1
K_2
K_3
K_8
q_1
q_2
q_3
q_N
$k_{1,1}$
$k_{2,1}$
$k_{3,1}$
$k_{8,1}$
$k_{1,2}$
$k_{2,2}$
$k_{3,2}$
$k_{8,2}$
$k_{1,8}$
$k_{2,8}$
$k_{3,8}$
$k_{8,8}$
mac_array.v
mac_col.v
qmem
SFU
SFU
SFU
SFU
ofifo
pmem

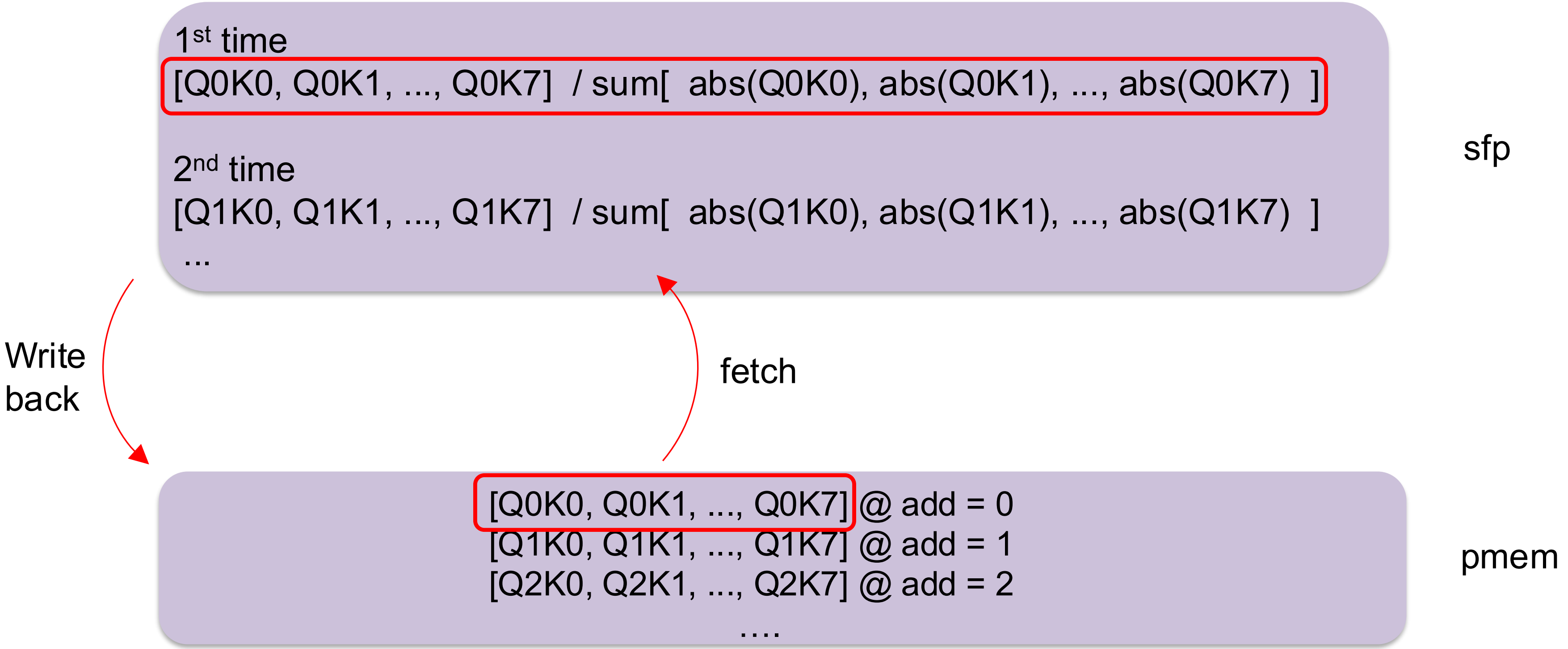


q_in[63:0]
i_inst[1:0]
mac_col
q_out
o_inst
out
fifo_wr

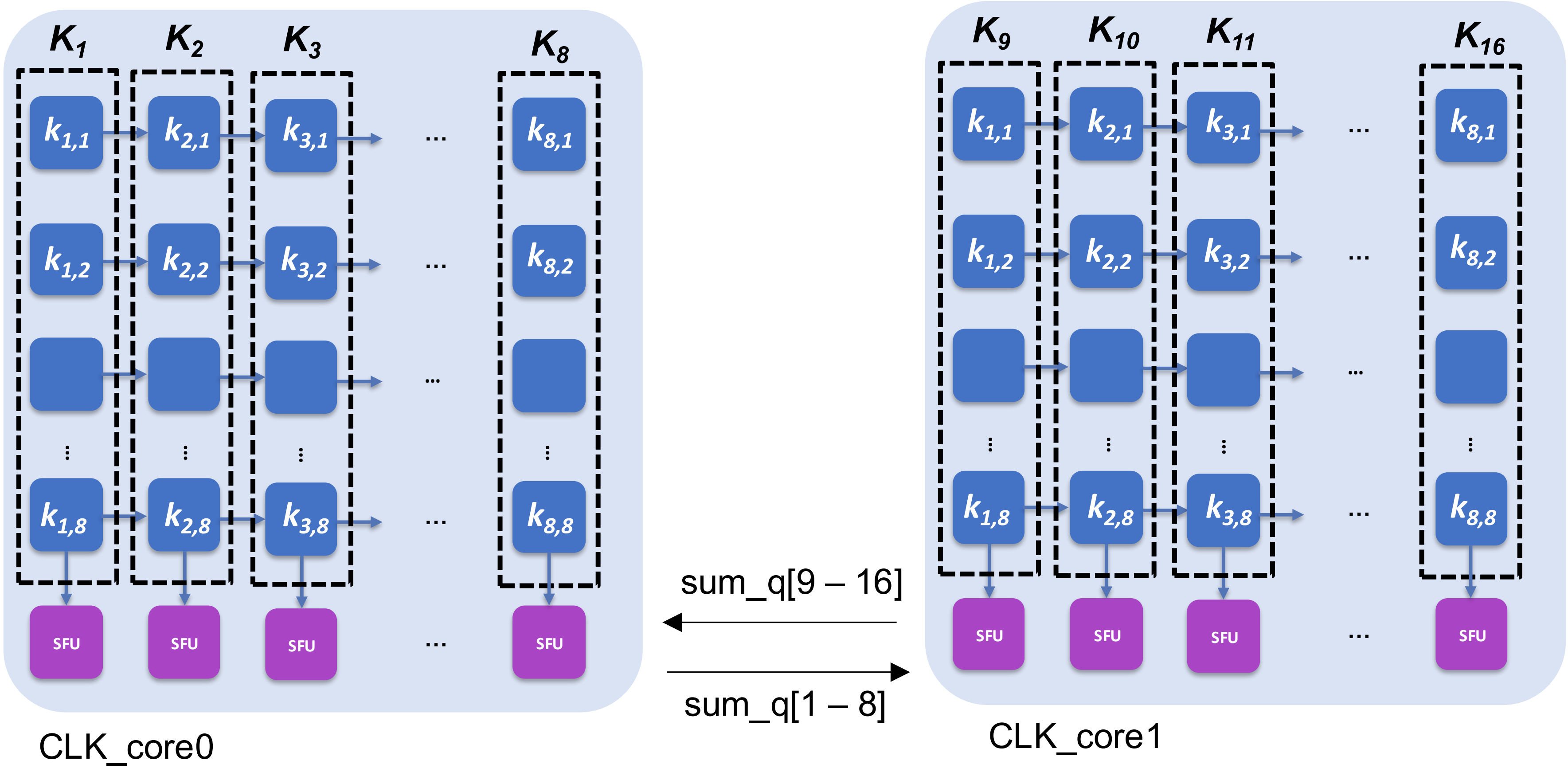
OFIFO Operation



Normalization



Extension for Dual Core



Normalize by "sum(abs(Q0K0), abs(Q0K1), ..., abs(Q0K7)) from core0
+ sum(abs(Q0K8), abs(Q0K9), ..., abs(Q0K15)) from core1"

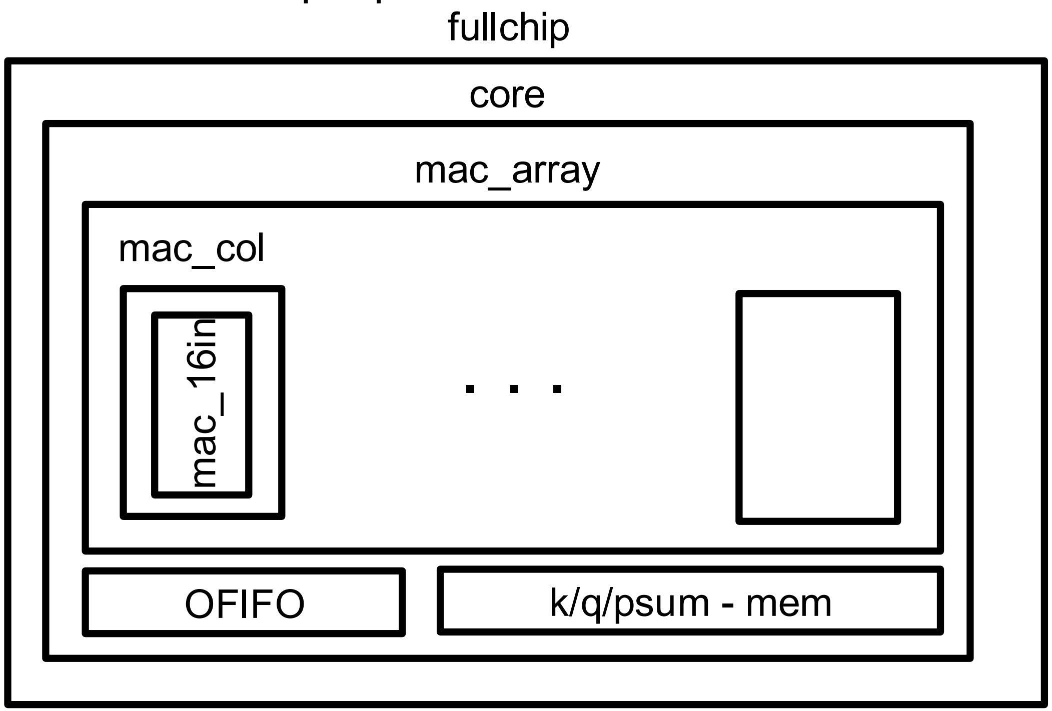
Optimization

- Optimize the implementation to maximize the frequency and minimize power
  - However, the project is not a spec competition
- Apply techniques you have learned from this course
- Potential optimization (does not mean you need to do everything below)
  - Pipeline, multi-cycle path, memory double-buffering, clock gating, loop unrolling, more # of cores, any other creative idea
  - Async interface handling (synchronizer, hand shaking, ….)
  - Maximize parallelism between processing stages
  - Design your own controller
  - better verification (with random input)
  - Minimize WNS and DRC errors

Understanding Verilog

- Current Verilog includes verification module inside
- Result (pmem_out) of core.v hasn't been connected to output port
- Instructions (fullchip_tb.v)

```verilog
assign inst[16] = ofifo_rd;
assign inst[15:12] = qkmem_add;
assign inst[11:8]  = pmem_add;
assign inst[7] = execute;
assign inst[6] = load;
assign inst[5] = qmem_rd;
assign inst[4] = qmem_wr;
assign inst[3] = kmem_rd;
assign inst[2] = kmem_wr;
assign inst[1] = pmem_rd;
assign inst[0] = pmem_wr;
```



How to prove ?

- Prove the efficacy of your idea (why ? how ? how much ?)

  - measure power and check functionality with VCD

    (SDF is not required, but it will be plus alpha)

  - clear comparison before vs. after

  - clear description in the report regarding your design choice

    e.g., why you chose the decision, why you cut the pipeline boundary, ….

  - report is VERY important

Project Policy

- TA & Instructor **will not**

  - solve your project instead of you

  - debug your code (e.g., I got this error with screenshot, can you check my code ?)

  - analyze your issue (e.g., my performance is not so good, do you know why ?)

  - comment on your result (e.g., Can you check my result is correct ?, Can I get A grade with this optimization ?)

  - “fighting against errors” is the biggest part of this project

- However, TA & Instructor will

  - clarify the project problem

  - help any environment issue (e.g., it seems there is no license file, tool does not load, …)

  - provide general guideline for a certain issue

- You can still discuss between students through Piazza